

# PilotFish Sandbox — Interface Construction Playbook

**Audience:** Cursor agents (and humans reviewing agent work).

**Scope:** Any new or substantially changed PilotFish interface under `Clients/`.

**Purpose:** One repeatable way to design, scaffold, implement, document, and smoke-test interfaces so demos stay consistent and production risks are called out honestly.

Do **not** invent a one-off layout. Follow this playbook, then specialize only where the integration requires it.

---

## How agents know how to build PilotFish interfaces

Cursor agents are **not** formally trained or certified by PilotFish. There is no special PilotFish fine-tune behind this Sandbox. Knowledge comes from a stack of sources when building an interface here:

### 1. Authoritative sources in this workspace

| Priority | Source                                                                                                                                                     | Use it for                                                                                                                                   |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| A        | External docs:<br><code>/Users/brianhannan/Documents/PilotFish Documentation</code> (see <code>PilotFish_Documentation/DOCUMENTATION_LOCATION.txt</code> ) | Module <b>inventory</b> , UI-type → class semantics, deep-dive behavior (see §1.1)                                                           |
| A        | <code>PilotFish_V2/</code> (XCS / module Java + <code>modules.conf</code> )                                                                                | <b>Same EIP module code V1 uses</b> — FQCNs, <code>ConfigurationItem</code> names/defaults, listener/processor/transport behavior (see §1.2) |
| B        | Working Sandbox demos under <code>Clients/**</code>                                                                                                        | Proven <b>wiring</b> of those modules into V1 <code>route.xml</code> on <code>pilotfish-eip:23R1</code>                                      |
| B        | This playbook + <code>.cursor/rules/</code>                                                                                                                | Construction process, V1 vs V2 policy, risks, definition of done                                                                             |
| B        | <code>EDI/TableData/</code> (see §3.6 + <code>EDI/README.md</code> )                                                                                       | Canonical X12 IG table data, XSDs, IG PDFs, and reference <code>examples/*.edi</code> for EDI demos                                          |
| C        | Your chat + <code>DESIGN.md</code>                                                                                                                         | Job-specific contracts and overrides                                                                                                         |
| D        | Live smoke tests / <code>logs/eip.log</code>                                                                                                               | Confirm the chosen module loads on the demo image                                                                                            |
| E        | General model knowledge                                                                                                                                    | Only to fill gaps after A–D                                                                                                                  |

**Important:** V2 does **not** invent a separate module stack. EIP modules (Listeners / Processors / Transports / Routing) are essentially the **same Java modules** whether the route document is classic V1 `route.xml` or V2 `route.v2.xml`. V2 mainly changes **how modules are connected and laid out**. Therefore agents **must** use `PilotFish_V2` and the external `PilotFish Documentation` project to decide which modules exist and how they are configured, then still **author runtime routes as V1** for this Sandbox's `eiPlatform` image unless the user overrides.

#### 1.1 External PilotFish Documentation (deep-dive library)

Canonical path (separate Cursor project; do **not** copy docs into this Sandbox):

`/Users/brianhannan/Documents/PilotFish Documentation`

Pointer in this repo: `PilotFish_Documentation/DOCUMENTATION_LOCATION.txt`

| Path (under the external project)                                                        | Role                                                                                 |
|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Documents/README.md                                                                      | Layout: Listeners / Processors / Transports × version (XCS, 23R1.127, 26R1.11, ...)  |
| Documents/General/Process/PilotFish-Module-Docs/Tracker-Interface-<br>(+ PDF if present) | <b>Master interface</b> — XML type, short class name, which versions have deep dives |
| Documents/Listeners/, Documents/Processors/, Documents/Transports/ (<version>/...)       | Deep-dive config reference for that module                                           |
| Documents/General/XCS/*                                                                  | Framework (threads, transactions) when designing concurrency / TX behavior           |
| Documents/General/Process/*                                                              | How docs were generated — rarely needed for interface build                          |

### How to use when constructing an interface:

1. Open the **tracker** in the external project and find the UI type you need (e.g. Directory / File, HL7 LLP, Database Polling (SQL)).
2. Note the **class** (DirectoryListener, DatabaseSqlListener, ...) and whether a deep-dive exists for a version close to the runtime image (pilotfish-eip:23R1 demos → prefer 23R1.\* docs when present; otherwise use 26R1.11 / XCS and call out version skew in DESIGN.md).
3. Read the deep-dive for config item names, defaults, and pitfalls **before** inventing XML.
4. If the tracker lists a module but there is **no** deep-dive yet, fall through to PilotFish\_V2 Java / modules.conf (§1.2), then a Sandbox example if any.

Do **not** skip Documentation when a relevant deep-dive exists.

### 1.2 PilotFish\_V2/ (module source + catalog — shared with V1)

Primary tree: PilotFish\_V2/pilotfishdevelopment-xcs-b313c66ccd5f/

| Path                                                             | Role                                                                 |
|------------------------------------------------------------------|----------------------------------------------------------------------|
| XCS-console-gui/.../modules.conf                                 | <b>Catalog of loadable module FQCNs</b>                              |
| subprojects/modules-*/src/main/java/com/pilotfish/eip/modules/** | Module definition — package + class = route.xml class="..."          |
| Configuration / descriptor types on those classes                | Exact <b>config element names</b> and defaults under <Configuration> |
| .clinerules/BLUEPRINT.md (or .aiassistant/rules/BLUEPRINT.md)    | V1 stage model vs V2 node model; what conversion should preserve     |
| V2_RouteViewer/                                                  | Viewer/reference assets for diagrams                                 |

### How to use when constructing an interface:

1. Search modules.conf and/or Java under com/pilotfish/eip/modules for the capability (e.g. SNIP, DirectoryListener, HL7TCP).
2. Resolve the **fully qualified class name** from the Java package + class (or the modules.conf line).
3. Read configuration item definitions (and nearby tests/resources) so XML tags match the code — **do not invent tag names**.
4. Prefer modules that also appear in Sandbox demos when you need a drop-in XML skeleton; otherwise scaffold from V2 + Documentation, then smoke-test on pilotfish-eip:23R1.
5. Use BLUEPRINT / importer notes when mapping V1 Source/Target/Router into generated route.v2.xml.

Wrong package / class strings have already broken demos. Treat FQCNs as **looked-up constants** from Documentation/V2 (or a working demo), never as creative writing.

### 1.3 Sandbox demos (Clients/\*\*) — proven assemblies

Best source for **end-to-end patterns** that already run in Docker:

| Demo                                     | Pattern                                                       |
|------------------------------------------|---------------------------------------------------------------|
| Clients/Demos/edi-837-snip-sqlserver/    | SQL poll → EIP handoff → fork → EDI + SNIP                    |
| Clients/Demos/edi-278-prior-auth/        | Completeness + simulated PA; EDI XML→EDI + HL7 XML→ER7 (§3.5) |
| Clients/Demos/hl7-healthcare-automation/ | Directory → validate → router fan-out → SQL + file            |

**Default assembly method:** choose modules via Documentation + V2 → copy a Sandbox `route.xml` fragment when one exists for that class → otherwise build V1 XML from looked-up config items → convert to V2 for diagrams.

### 2. How agents pick processors / class names (`route.xml`)

Decision order:

1. **Identify the capability** (directory listen, SQL poll, XSLT, SNIP, HL7 LLP, ...).
2. **External docs tracker** (/Users/brianhannan/Documents/PilotFish Documentation/Documents/...) → UI type + class; read deep-dive if present.
3. **PilotFish\_V2 modules.conf + Java** → confirm FQCN and configuration item XML names.
4. **Sandbox demo route.xml** → copy a working module block when available.
5. **Smoke test** on the demo EIP image; fix using logs if the class is missing from that WAR (docs/V2 may be newer than 23R1).
6. **Ask the user** if the module isn't in the image and can't be substituted.
7. **General model knowledge** only as a last-resort hint — never the source of an FQCN.

### 3. V1 vs V2 — runtime vs diagrams

| Artifact                                                                                                                                     | Format                                | Role                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|---------------------------------------------------------------------|
| pilotfish/demo-eip-root/**/route.xml                                                                                                         | <b>V1</b>                             | <b>Runtime</b> for pilotfish-eip:23R1 — <b>author here</b>          |
| eip-root/**/route.v2.xml + modules/*.xml                                                                                                     | <b>V2</b>                             | Routes tab / docs / PDF — usually <b>generated</b> from V1          |
| tools/convert_routes_to_v2.py                                                                                                                | V1 → V2                               | Keeps <code>route.xml</code> ; emits diagram graph                  |
| PilotFish_V2/                                                                                                                                | Product + <b>shared module source</b> | Module catalog & behavior <b>and</b> V2 format/editor understanding |
| /Users/brianhannan/Documents/PilotFishDeep dives + tracker Documentation/Documents/ (see PilotFish_Documentation/DOCUMENTATION_LOCATION.txt) |                                       | Interface creation decisions (which module, which settings)         |

**Policy:** Use V2 **code** and Documentation to choose and configure modules; still **ship V1 route.xml as runtime** and generate V2 for visualization, unless the user explicitly wants native V2 runtime.

### 4. Instructions you give in chat

Your prompts, corrections, ports, case-study links, and overrides win over defaults. `DESIGN.md` is the working spec for that interface.

## 5. General model knowledge (not PilotFish training)

Horizontal knowledge (integration patterns, HL7/EDI concepts, Docker, SQL, Flask) — **not** a substitute for the external PilotFish Documentation project or PilotFish\_V2 when picking modules.

## 6. External / fetched material (when used)

Public case studies and links you provide; live smoke evidence. Prefer in-repo Documentation + V2 over stale web pages when they conflict on config item names.

## 7. What is *not* a source of authority

- Invented FQCNs or config element names
- “Only demos may be used” as a hard ceiling when Documentation/V2 clearly provide a better module (demo copying is preferred for **skeletons**, not an excuse to ignore the catalog)
- Claiming compliance without fail-closed config
- Assuming Documentation 26R1 deep-dives always match `pilotfish-eip:23R1` without smoke test
- Hand-maintained V2 as the only runtime truth for current Sandbox demos

## Practical rule of thumb

Your chat + DESIGN.md

- External PilotFish Documentation project (tracker + deep dives)
- PilotFish\_V2 (modules.conf + module Java = same modules as V1)
- nearest Sandbox demo route.xml (assembly skeleton)
- author V1 runtime → convert to V2 for viewer/PDF
- smoke test on `pilotfish-eip:23R1`

If those disagree, **prefer V2/docs for module identity + Sandbox/live smoke for “loads on this image”**, and document version skew in DESIGN.md.

---

## 0. When this applies

Use this playbook when asked to:

- Create a new PilotFish demo / interface / route stack
- Port a case study into a runnable Sandbox demo
- Add a route, listener, transform, or transport to an existing Sandbox interface
- Produce a design document for an interface

If the task is a tiny fix (typo, port conflict, one SQL row), skip the full scaffold and only apply relevant sections (smoke test, honesty about validation, no secret sprawl).

---

## 1. Pre-flight (block implementation until answered)

Capture answers in the interface DESIGN.md (see §8). If missing, ask the user.

| # | Question                                                                            | Why                                |
|---|-------------------------------------------------------------------------------------|------------------------------------|
| 1 | <b>Business job</b> in one sentence?                                                | Keeps scope honest                 |
| 2 | <b>Inbound contract</b> (format, envelope, transport: dir / MLLP / SQL / API)?      | Wrong listener = silent no-ops     |
| 3 | <b>Outbound contract(s)</b> (file / SQL / EIP / HTTP) and <b>success criteria</b> ? | Dual-write needs an explicit model |

| # | Question                                                                          | Why                                       |
|---|-----------------------------------------------------------------------------------|-------------------------------------------|
| 4 | <b>Validation</b> level (none / heuristic / SNIP / HL7 structural / partner ACK)? | Don't claim compliance without gates      |
| 5 | <b>Identity &amp; idempotency</b> keys?                                           | Dedup / retries                           |
| 6 | <b>Happy-path sample(s)</b> available?                                            | Seed data drives the first green path     |
| 7 | <b>Demo vs production labeling</b> — is this demo-only?                           | Secrets, SA, PHI mounts                   |
| 8 | <b>Ports</b> free on this machine?                                                | Avoid collisions with other Sandbox demos |

**Port allocation (default pattern):** pick an unused triple near existing demos:

| Role            | Existing examples        |
|-----------------|--------------------------|
| SQL Server host | 14335 (EDI), 14336 (HL7) |
| EIP host        | 8093 (EDI), 8096 (HL7)   |
| Web UI host     | 8095 (EDI), 8097 (HL7)   |

Document chosen ports in `README.md` before `compose up`.

### 1.1 LAN access (required when spinning up a Web UI)

Whenever an interface Web UI is brought up for local demo / review (`docker compose up`, smoke test, or “spin it up”), agents **must** expose it on the machine’s **192.x LAN address** as well as `localhost`, so it can be opened from phones, tablets, and other devices on the same network.

#### Required every time:

1. Detect the host’s primary IPv4 on a `192.*` interface (this Sandbox’s usual address is `192.168.68.52` on `en0`; re-detect — do not hardcode forever if DHCP changes).
2. Publish the Web UI with Docker host port mapping (already binds `0.0.0.0` by default) — do **not** bind only to `127.0.0.1`.
3. Set compose env `LAN_HINT=http://<192.x.x.x>:<webui-port>/` so the UI nav shows the LAN link.
4. List **both** URLs in `README.md` and in the agent summary when the stack is running:
  - Local: `http://localhost:<port>/`
  - LAN: `http://192.x.x.x:<port>/`
5. Prefer Flask/Web UI `host=0.0.0.0` (already the demo pattern).

**Anti-pattern:** shipping or announcing only `localhost` / `127.0.0.1` when the user needs to review from another device.

For **PDFs and other review files**, also follow §6.2 (serve under `/documents/...` and paste LAN browser links — do not rely on Google Drive alone).

## 2. Standard directory scaffold

Create under `Clients/Demos/<slug>/` (or `Clients/<Client>/<slug>/` for client work):

```
<interface>/
  README.md
  DESIGN.md                # filled from §8 template
  docker-compose.yml
  sql/
```

```

    01_init.sql                # idempotent preferred; document if destructive
samples/                      # inbound fixtures (named clearly)
input/                        # runtime drop dirs (gitkeep / .gitkeep)
output/                       # artifacts (edi, snip, clearinghouse, kickout, archive...)
eip-root/                     # diagram / V2 source of truth for route viewer
    interfaces/<App>/routes/<N - Name>/
        route.xml
        route.v2.xml          # generate/convert; don't hand-edit unless required
        modules/*.xml
pilotfish/
    demo-eip-root/            # what the container actually mounts at runtime
    Dockerfile                # FROM pilotfish-eip:23R1 (+ JDBC if needed)
    demo-entrypoint.sh
    conf/environment-settings.conf
webui/                         # Flask (or existing demo UI pattern)
    app.py
    templates/
    static/
        route-viewer/        # copy from an existing demo; keep in sync
tools/
    convert_routes_to_v2.py    # if needed
    export_route_diagrams.py   # PDF: overview (collapsed groups) + detail pages → documents/
    export_stakeholder_brief.py # stakeholder Capability Brief PDF → documents/
    export_test_plan_pdf.py    # Test Plan PDF from tests/plan.json
    export_test_results_pdf.py # (optional wrapper) results PDF helper
    run_interface_tests.py     # execute plan → documents/test-results.json|.html|.pdf
    post_up_tests.sh           # compose-up helper: wait + run tests
    scrub_pdf_secret_false_positives.py # optional: clear GitHub Vault FP in PDF binaries
tests/
    plan.json                 # living automated test plan (source of truth)
documents/                   # required: route PDF + capability brief + test plan (+ results)
    <Interface>_V2_Route_Diagrams.pdf
logs/                        # gitignored runtime logs if bind-mounted

```

Under each route directory that has a long processor chain, also keep:

```

eip-root/.../routes/<N - Name>/
    route.xml
    route.v2.xml
    modules/*.xml
    diagram-groups.json        # docs-only Processor Groups (see §6.1b)

```

Mirror `diagram-groups.json` into `pilotfish/demo-eip-root/routes/<...>/` when that tree is what compose/Web UI mounts (or ensure the Web UI `ROUTES_DIR` mount points at the tree that contains the JSON).

**Source of truth:** runtime behavior = `pilotfish/demo-eip-root`.

**Diagrams / Routes tab** = `eip-root` (or a documented sync job).

**Route design PDF** = `documents/` (generated; do not leave only under `output/route-diagrams`).

If runtime and diagrams diverge, fix or document the divergence in `DESIGN.md` — never leave silent drift.

Reference implementations:

- `Clients/Demos/edi-837-snip-sqlserver/` — SQL poll → EIP handoff → fork → EDI XML→EDI + SNIP
- `Clients/Demos/edi-270-271-eligibility/` — EligibilityRequest → EDI XML → EDITransformationProcessor (XML→EDI) → 270

- Clients/Demos/edi-278-prior-auth/ — AuthDecision → EDI XML→EDI (278) + HL7 XML→ER7 (ORU); no hardcoded wire text
  - Clients/Demos/hl7-healthcare-automation/ — directory listen → validate → router fan-out → SQL + file
  - Clients/Demos/fhir-r4-platform/ — Call Route auth, docs-only **Processor Groups** (diagram-groups.json + overview/detail route PDF)
  - Med Rec Flat File to HL7 (eip-root/interfaces/Flat File to HL7 and Kickout Reports/) — XSLT HL7 XML → HL7TransformationProcessor (XML→HL7 2.X)
- 

### 3. Route construction pattern

#### 3.1 Canonical stage chain (V1 runtime)

Author this chain in **V1** route.xml (what pilotfish-eip:23R1 runs):

Listener

- Source processors (normalize / validate / map / snapshot files)
- FormatProfile (relay | XPath fork when true multi-message)
- XPathRoutingModule (Conditional Router)
- Target processors (pre-transport map)
- Transport(s) # Directory | DatabaseSql | EIP | HTTP ...

Then run tools/convert\_routes\_to\_v2.py (or demo equivalent) so route.v2.xml stays aligned for the Routes tab / PDF. Do **not** hand-maintain V2 as the only copy of runtime truth.

#### 3.2 Naming

- Routes: 1 - <Verb Object>, 2 - ... (same spirit as existing demos)
- Modules: stable UUIDs once created; don't regenerate casually (breaks V2 links)
- Env props: \$\$\$sqlserver.\*, \$\$\$\*\_DIRECTORY in environment-settings.conf
- Route viewer / PDF: resolve \$\$\$NAME
- Web UI **XSLT** tab: list/view every .xsl / .xslt under routes/ (hide the tab when none exist). API: /api/v2/xslt, /api/v2/xslt/content?path= from that conf for display; **mask** names matching password/secret/token/apiKey/privateKey (show .....). Directories, drivers, URLs, usernames remain visible.

#### 3.3 Must-design decisions (write into DESIGN.md)

##### 1. When is business state committed?

Never mark work terminal (PROCESSED / archived-as-success) before all required side effects succeed — or document that the demo deliberately does claim-before-complete and list it under Risks.

##### 2. Dual / multi write:

If router fans out to SQL **and** files (or multiple files), define order + compensation (outbox, or “file then SQL”, or accept demo risk explicitly).

##### 3. Fork vs snapshot:

A stage named “Split” / “Batch” must either truly fork (XPathForkingModule / equivalent) or be renamed (e.g. “Batch Flag Snapshot”). No fake splits.

##### 4. Validation gate:

If SNIP / HL7 / XSLT “validation” runs with StopOnError=false and always continues, say so. Prefer fail-closed kickout + non-success status for anything marketed as validation.

##### 5. Validate the wire artifact:

Don't SNIP / schema-validate a pre-transform or stripped XML if the partner receives the post-transform payload.

#### 6. **Batching / back-pressure:**

SQL listeners: bounded TOP (@n) + locking hints when claiming rows.

Triggerable / forked routes: throttling when heavy processors (SNIP) sit downstream.

#### 7. **Idempotency:**

Unique business keys in SQL; versioned or non-overwrite filenames for audit outputs.

#### 8. **Poison / knockout:**

Explicit fail directory or status; don't Move-to-archive on exception-only paths.

#### 9. **EDI / HL7 emission:**

Outbound X12 or HL7 v2 **must** use structured PilotFish EDI/HL7 XML + the matching Transformation processor (§3.5). Hardcoded ISA\* / MSH| text in XSLT is not allowed for new work.

### 3.4 Prefer PilotFish routes/interfaces over custom modules (default)

**Bias: build it as a PF route (or callout route) first.** Custom Java modules (custom-modules/\*, fat JARs into WEB-INF/lib) are a **last resort**, not a convenience shortcut.

| Prefer a PF interface/route when...                                                                                                                                       | A custom module may be justified when...                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Stock listeners/processors/ transports can assemble the behavior (Call Route, HTTP Post/Form Post, Attribute Population, RegEx, XSLT, SQL, Directory, Sync Response, ...) | Crypto/protocol libs or HAPI/third-party APIs cannot be expressed without unreasonable route spaghetti            |
| Behavior is orchestration, auth callouts (e.g. OAuth introspect), mapping, routing, persistence                                                                           | Performance requires in-process compiled logic that is measured and documented                                    |
| Another engineer should edit it in eiConsole without a Java rebuild                                                                                                       | The PF graph would be so large/confusing that a small module is clearer <b>and</b> DESIGN.md records why          |
| Demo honesty improves by showing real PilotFish topology                                                                                                                  | No stock module exists in the runtime image and no route substitute works after Documentation/PilotFish_V2 search |

#### Required process before adding a custom module:

1. Search Documentation tracker + PilotFish\_V2 + Sandbox demos for a stock module or callout-route pattern.
2. Sketch the PF-only design (including a dedicated callout route if mid-pipeline HTTP/auth is needed).
3. Only if that design is **extremely difficult**, confusing beyond salvage, or impossible on pilotfish-eip:23R1, implement a custom module.
4. Document in DESIGN.md: why PF-only was rejected, what the module owns, and how to remove it later if PF catches up.
5. Keep the custom surface minimal; prefer route-owned wiring around a tiny processor over a mega-module.

**Good example (this Sandbox):** FHIR Keycloak auth was moved from FhirJwtAuthProcessor to route 0 - Keycloak JWT Auth (Call Route + HTTP Post introspection). Validation/Bulk may remain custom while HAPI/Bulk semantics stay hard to express in stock modules alone.

### 3.5 Emit X12 / HL7 via PilotFish transform modules (required)

When an interface **generates** X12 EDI or HL7 v2 wire (outbound file, HTTP body, LLP payload, response envelope), **do not** build the wire as concatenated text in XSLT (ISA\*..., MSH|..., xsl:output method="text" with delimiters). That hides PilotFish's product value and is harder to maintain.

#### Required pattern



Business / decision XML

- XSLT (method="xml") builds PilotFish structured EDI XML or HL7 XML
- EDITransformationProcessor (TransformationDirection = "XML to EDI")  
or HL7TransformationProcessor (TransformationDirection = "XML to HL7 2.X")
- transport / file write (optional EOLProcessor for HL7 LF)

| Standard       | Intermediate XML                                                                                                                             | Transformation processor                                                  | FQCN                   |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|------------------------|
| <b>X12 EDI</b> | XCSDData → Interchange → Group → Transaction with <b>named segments</b> (ST, BHT, NM1, ...) and composites (SV101/SV101_1, HI01/HI01_1, ...) | EDITransformationProcessor · XML to EDI                                   | com.pilotfish.eip.modu |
| <b>HL7 v2</b>  | Message root (ORU_R01, ADT_A01, ...) with MSH.1/MSH.2/MSH.9/MSG.*, PID.*, OBX.*, datatype components (CE.1, XPN.1, ...)                      | HL7TransformationProcessor · XML to HL7 2.X · set HL7Version (e.g. 2.5.1) | com.pilotfish.eip.modu |

### XSLT rules for these maps

- Output **XML only** (xsl:output method="xml"). Populate structure and business values; let the transform module own delimiters, envelopes, and segment encoding.
- Include an explicit <ST>...</ST> on modern X12 outbound XML (23R1 does **not** auto-inject ST from @DocType without legacy shape).
- Prefer UseInternalData=false + UseProvidedDelimiters=true **and** Sandbox EDI/TableData via USE\_ENHANCED\_CONTEXT=true + TransactionDataWithVersion (§3.6) on pilotfish-eip:23R1 (trial X12 tables are expired). Call the chosen IG folders out in DESIGN.md.
- For HL7: USE\_FRIENDLY\_NAMES=false, USE\_NAMESPACE=false, match MSH.12 to HL7Version. Optional EOLProcessor (EndlineSequence=\n) when file consumers prefer LF.
- Write intermediate EDI/HL7 XML to a debug/audit folder in demos so the XSLT tab and reviewers can see the structured map.

### Allowed exceptions (must be labeled in DESIGN.md Risks)

- Pure **relay** of partner-supplied wire already in the stream (no generation).
- Tiny **demo shims** outside EIP (e.g. Web UI ST inject) when the route path itself still uses the transform module.
- User explicitly waives the rule for a throwaway spike.

### Reference assemblies

- X12 XML→EDI: Clients/Demos/edi-270-271-eligibility/, Clients/Demos/edi-837-snip-sqlserver/, Clients/Demos/edi-278-prior-auth/ (278 response)
- HL7 XML→ER7: Med Rec / Flat File to HL7 (eip-root/interfaces/Flat File to HL7 and Kickout Reports/), Clients/Demos/edi-278-prior-auth/ (ORU notice)
- Structured dialect docs: /Users/brianhannan/Documents/PilotFish Documentation/Documents/Processors/.../Pi

**Anti-example (do not copy for new work):** XSLT that emits ISA\*00\*... or MSH|^~\&|... as text. Legacy demos that still do this (hl7-healthcare-automation event→HL7 text, older doc-healthcare maps) are **not** the construction standard going forward—refactor when touching those emitters.

### 3.6 Sandbox EDI/TableData — use it whenever it improves the demo (required for X12)

Canonical tree (sibling of docs/): EDI/TableData/x12/ — see EDI/README.md.

This is the Sandbox’s licensed-style WPC implementation-guide **table data** (plus XSDs, IG PDFs, and **examples/\*.edi**). On **pilotfish-eip:23R1**, bundled **trial** X12 table data is expired, so demos must **not** rely on **UseInternalData=true** alone.

#### When building or touching any X12 interface under **Clients/**:

1. **Mount** **EDI/TableData/x12** into EIP at  
`/usr/local/tomcat/webapps/eip/eip-root/edi-tabledata`  
(compose volume, read-only). Existing EDI demos already do this.
2. On every **EDITransformationModule** / **EDITransformationProcessor** that parses or emits that IG:
  - **UseInternalData=false**
  - **USE\_ENHANCED\_CONTEXT=true**
  - **TransactionDataWithVersion** pointing at the matching IG folder(s) under **edi-tabledata/**, version **5010**, using  
`{ognl:@com.pilotfish.utils.PilotFishUtils@getWorkingDirectory()+'/edi-tabledata/<IG>'}`  
Example for 835: `.../edi-tabledata/835-W1`. Multiple IGs: concatenate multiple `[eip_pair:...:eip_name:5010:e` entries (e.g. `270-A1 + 271-A1`, or `278-A1 + 278-A3`).
3. **Prefer** IG **examples/\*.edi** (and XSDs/PDFs) from **EDI/TableData** for reference fixtures — copy selected files into the demo’s **samples/tabledata/** when useful for UI/file drops. Keep **story-specific** happy-path samples under **samples/** when the demo logic depends on crafted content (underpays, completeness matrix, etc.).
4. Keep inbound XPaths dual-friendly (`//CLP/CLP01 | ... | //Segment[Element[1]='CLP']/...`) when migrating, so named-segment XML and basic EDI XML both work during transition.
5. Call the **TableData** mount + chosen IG folders out in **DESIGN.md** (replaces “trial tables expired → basic EDI XML only”).

**Do not** invent ad-hoc second copies of WPC table trees outside **EDI/TableData** unless the user asks. Update the canonical tree, then remount.

**Reference demos (already wired):** **edi-835-payment-integrity**, **edi-835-oci-bucket**, **edi-278-prior-auth**, **edi-270-271-eligibility**, **edi-270-271-realtime**, **edi-837-snip-sqlserver**.

---

## 4. Implementation order (agent checklist)

Work in this order unless the user specifies otherwise:

0. **Start timing (§4.1):** create/update **documents/build-timing.json** with **started\_at** (UTC) as soon as the user asks for a new demo.
1. **DESIGN.md** filled from §8 (can be draft; must exist before claiming “done”).
2. **Module selection** via external PilotFish Documentation tracker/deep-dives + **PilotFish\_V2** (**modules.conf** / module Java). Record chosen FQCNs in **DESIGN.md** Pipeline table. **Do not invent a custom module** until §3.4 is satisfied.
3. **SQL init + samples** (if applicable).
4. **environment-settings.conf + compose + Dockerfile** (ports, volumes, heap if SNIP).
5. **Minimal Route 1 V1 route.xml** (listener → snapshot/file) green — copy Sandbox skeleton when possible. Prefer callout routes (Call Route + TriggerableListener) for reusable auth/utility steps.
6. **Downstream routes / transforms** one stage at a time (still V1 runtime).
7. **Router + transports** with kickout path tested once.
8. **Web UI** inject/submit + status views matching the demo story, including the standard **Info** tab (§6.5) with PDF aliases.
9. **V2 convert** into **eip-root + Routes** tab / docs.
10. **Route design PDF (required):** with Web UI up, author **diagram-groups.json** for long chains (§6.1b), then run `python3 tools/export_route_diagrams.py --config compact` (or

--config changed) so documents/ gets an **overview (collapsed groups) + later detail pages** PDF (see §6 / §6.1b). 10b. **Stakeholder Capability Brief PDF (required):** run `python3 tools/export_stakeholder_brief.py` so `documents/<ShortName>Capability_Brief.pdf` is generated from `DESIGN.md` + routes (+ `CapabilityStatement` when FHIR). See §6.1a. 10c. **Test plan PDF + automated run (required):** maintain `tests/plan.json`, run `python3 tools/export_test_plan_pdf.py` and `python3 tools/run_interface_tests.py --wait` (see §7.1). Update the plan as capabilities are added.

11. **README.md** (run, ports, smoke commands; link to `documents/*.pdf`).
12. **Smoke / automated tests** (§7 / §7.1) and paste pass/fail summary into the chat (and update `DESIGN.md` Risks if findings).
13. **Finalize timing + Docker inventory (§4.1 / §5.1):** complete `build-timing.json`, run `python3 tools/list_sandbox_demo_docker.py`, retain Hindsight note when the user marks the demo done.

Do not refactor unrelated demos. Prefer copying the closest existing demo assembly after modules are chosen from Documentation/V2.

#### 4.1 Build timing — track start, end, and slow phases (required)

Goal: measure **end-to-end demo construction time** (first user ask → user-deemed complete) and learn which phases take longest so the process can get faster.

**When it starts:** the moment the user asks to create / scaffold / build a new interface demo (or a major net-new stack). Record wall-clock UTC immediately — do not wait until design is finished.

**When it ends:** when the user says the demo is complete **or** explicitly accepts Definition of Done (§11). If unclear, ask. Do not invent an “end” silently after smoke tests alone.

**Artifact (per demo):** `documents/build-timing.json`

Schema: copy `docs/templates/build-timing.example.json`. Keep updating the file during the build (phases can be filled as you go; finalize on completion).

| Field                                                            | Purpose                                                                     |
|------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>started_at</code> / <code>completed_at</code>              | ISO-8601 UTC; drive <code>duration_minutes</code>                           |
| <code>phases[]</code>                                            | Named stages with start/end + duration (map to §4 checklist where possible) |
| <code>slowest_phases</code>                                      | Sorted by duration — what to optimize next                                  |
| <code>bottlenecks</code>                                         | Concrete blockers (wrong FQCN, OGNL, rebuild loops, ...)                    |
| <code>speedup_ideas</code>                                       | Actionable process fixes for the next demo                                  |
| <code>compose_project</code> + <code>docker_at_completion</code> | Tie timing to §5.1 inventory                                                |

**Required phases to capture** (merge/split if the work is shorter, but cover these concepts):

1. Pre-flight + DESIGN
2. Scaffold (compose / SQL / ports)
3. Routes + transforms (often the long pole)
4. Web UI / inject / kickouts
5. V2 convert + route/capability/test PDFs
6. Smoke / automated tests + fixes

**Agent duties:**

1. Create `documents/build-timing.json` early with `started_at`.
2. Update phase timestamps as work progresses (approximate is OK; prefer honest ranges over precision theater).
3. On completion: set `completed_at`, compute durations, fill `slowest_phases`, `bottlenecks`, `speedup_ideas`, and a §5.1 Docker snapshot.
4. Mention total minutes + top 1–2 slow phases in the completion chat summary.
5. Retain a short Hindsight note for the bank `PilotFish-Sandbox` when a demo is marked complete (document id like `build-timing-<slug>`), so future sessions can recall bottlenecks.

Do **not** pad times or skip recording after a painful debug loop — those are the highest-value samples.

---

## 5. Runtime / Docker conventions

- Base image: `pilotfish-eip:23R1` (build from Sandbox root if missing).
- Bind-mount runtime interfaces and `input/` / `output/` / `logs/` as other demos do.
- Heap: if SNIP (or similarly heavy) processors load, set sufficient `CATALINA_OPTS` (EDI demo uses ~6GB max) and document why.
- Web UI **depends\_on** EIP when useful; document cold-start wait (often 60–90s).
- **LAN:** set `LAN_HINT` to `http://<192.x.x.x>:<webui-port>/` (see §1.1). Host ports must remain reachable on the LAN interface; announce both localhost and LAN URLs after `compose up`.
- Secrets: demo password may match existing demos for local convenience, but **DESIGN.md and README must say demo-only** — never invent “production-ready” claims around `sa` + shared passwords.
- Prefer least-privilege DB users when creating new non-demo client interfaces.
- **Compose project name:** prefer the demo folder slug (Compose default). Record it in `DESIGN.md` Ops and in `documents/build-timing.json`.

### 5.1 Sandbox demo Docker inventory (required during builds)

Goal: know **how many demo stacks/containers this project is keeping alive** so idle demos can be stopped without guessing.

**Inventory command (repo root):**

```
python3 tools/list_sandbox_demo_docker.py
# machine-readable:
python3 tools/list_sandbox_demo_docker.py --json
```

Reports:

- Compose projects whose config path is under `Clients/`
- Running containers named `pf-*` or images containing `pilotfish`
- Demo dirs that have `docker-compose.yml` but are **not** in `docker compose ls` (idle candidates)

**When to run:**

1. **Start** of a new interface build (baseline).
2. **End** of a build (counts go into `build-timing.json` → `docker_at_completion`).
3. Anytime the user asks about Docker / disk / “what demos are running”.

In the **agent summary**, include e.g. “Sandbox: N Compose projects / M containers running” and name this demo’s project + container prefixes.

**Cleanup:** never `docker compose down -v` or prune images without the user’s OK. Prefer suggesting `cd <demo> && docker compose down` for stacks they are not using. Do not touch non-Sandbox Compose projects (e.g. unrelated folders on the machine).

---

## 6. Web UI, route viewer, and PDF

When the interface has routes (every new Sandbox interface with a route stack):

1. Copy `webui/static/route-viewer/` from a **current** demo that already has Processor Groups support (`fhir-r4-platform` is the reference: includes `diagram-groups.js`).
2. Serve `route.v2.xml` + module XML via `/api/v2/routes...` (mirror existing Flask helpers).
3. Also serve `GET /api/v2/routes/<route_id>/diagram-groups.json` (return `{ "groups": [] }` when the file is absent).
4. Routes tab: route select, layout dropdown, link to docs.
5. Docs / print view and `tools/export_route_diagrams.py` must use the same **Box config** modes:
  - **compact** — names only (**preferred default for committed route PDFs**; smaller, fewer secret-scanner false positives)
  - **changed** — non-default values
  - **all** — all ModuleConfig values
6. Export script: `python3 tools/export_route_diagrams.py --config compact`  
Screenshot URL must include `mode=docs&bare=1&config=<mode>`, and for grouped captures add `groups=1` plus either `collapse=all` (overview) or `group=<id>` (detail) — see §6.1b.
7. PDF and on-screen docs view should show equivalent config detail (same default).
8. After writing large route PDFs, optionally run `python3 ../../tools/scrub_pdf_secret_false_positives.py documents/*.pdf` (or `--demos`) so GitHub does not flag Vault-like false positives in compressed image streams.

### 6.1 Route design PDF → documents/ (required for every new interface)

Any time you **create a new interface** (or add/change routes enough that diagrams should be refreshed), you **must** generate the route design PDF automatically as part of finishing the work — do not wait for the user to ask.

1. Ensure Web UI is up and V2 routes render.
2. Author Processor Groups for long chains (§6.1b) before capturing.
3. Run `python3 tools/export_route_diagrams.py --config compact` (use `--config changed` only when stakeholders need inline config on the detail pages).
4. Write a **single** PDF under the interface’s **documents/** folder (no `_changed` suffix, no second copy):
  - `documents/<ShortName>_V2_Route_Diagrams.pdf`
5. PDF layout: **no cover page**; green brand header (and route/section title) on **every** page. Order captures as: short flat routes → **route overview (collapsed groups)** → **one detail section per group** → next route...
6. Expose the PDF from the Web UI at `/documents/route-diagrams.pdf` (mount `./documents` read-only; Routes tab link **Route design PDF**).
7. Mention the PDF path in `README.md` and announce browser URLs per §6.2.

Skipping the PDF requires the user to opt out explicitly; “optional” is not the default.

### 6.1b Processor Groups for long diagrams (required when chains are long)

PilotFish V2 `route.v2.xml` in this Sandbox is a **flat** node graph (no nested Group schema). Do **not** invent runtime Group XML or change V1 `route.xml` just for prettier PDFs.

Instead, every interface with a **long processor chain** uses a **docs-only** grouping layer:

| Piece                                                                  | Role                                                                                                                                                    |
|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>diagram-groups.json</code> next to <code>route.v2.xml</code>     | Declares logical groups by <b>processor/transport display labels</b>                                                                                    |
| <code>webui/static/route-viewer/diagram-groups.js</code><br>URL params | Collapses or focuses groups in the viewer<br><code>groups=1&amp;collapse=all</code> (overview) ·<br><code>groups=1&amp;group=&lt;id&gt;</code> (detail) |
| <code>export_route_diagrams.py</code>                                  | Captures <b>overview first</b> , then <b>one screenshot per group</b> into the same PDF                                                                 |

#### When groups are required

- Any Source or Target processor list with 4 processors in a row, **or**
- Any linear worker route with 8 processors total.

Short routes (e.g. 1–3 modules) may stay flat with no `diagram-groups.json`.

#### Authoring `diagram-groups.json`

```
{
  "groups": [
    {
      "id": "ingress",
      "title": "Ingress",
      "description": "Normalize, validate, auth",
      "labels": ["Save Raw Body", "...", "Call Auth Route"],
      "transports": ["Optional Sync Response Name"]
    }
  ]
}
```

Rules:

- Match nodes by **exact label** (post-module-load name / processor `name=` in V1). Re-run V2 convert then verify labels if rename breaks matches.
- Prefer **logical** groups (Ingress, Create, Kickoff, Per-type NDJSON Export, Complete Job) — not one box per single processor.
- Put **only processors** in labels. List related sync response transports under **transports** so detail pages can show **group → transport** without stuffing the transport into the collapsed box.
- Keep groups 2 members (the viewer skips singleton groups).
- Mirror the JSON into whichever tree the Web UI mounts as `ROUTES_DIR` (`eip-root/.../routes` and/or `pilotfish/demo-eip-root/routes`).
- Do **not** hand-edit `route.v2.xml` to fake nest nodes — converter will wipe it.

#### Viewer / export behavior (must keep working on every new demo)

- Copy `diagram-groups.js` + CSS for `.route-node-group` from `fhir-r4-platform` when scaffolding a new Web UI.
- Overview capture URL shape:

`/static/route-viewer/index.html?route=<id>&mode=docs&layout=pipeline&bare=1&config=compact&groups=1&col`

3. Detail capture URL shape:

...&groups=1&group=<group-id>

4. Export list order (example):

```
0 - Auth (flat if short)
1 - REST Platform (Overview)    ← collapse=all
1 · Ingress                     ← group=ingress
1 · Create                      ← group=create
...
3b - Worker (Overview)
3b · Per-type NDJSON Export
3b · Complete Job
```

5. Collapsed box UX (stakeholder-readable): dashed border, title **Processor Group**, short description, footer like `N processors · see detail page`.

6. Detail pages start with a **Processor Group · Detail** banner, then the full processor chain (+ transports when listed).

**Reference implementation:** Clients/Demos/fhir-r4-platform/ (diagram-groups.json on routes 1 / 3 / 3b, export\_route\_diagrams.py CAPTURE list, Web UI API).

### 6.1a Stakeholder Capability Brief PDF → documents/ (required for every new interface)

Any time you **create a new interface** (or change scope/capabilities enough that stakeholders should see an update), you **must** generate a higher-level Capability Brief PDF automatically — do not wait for the user to ask. This is the document shared with business / clinical stakeholders; the route diagrams PDF remains the technical wiring view.

1. Ensure DESIGN.md is current (purpose, actors, in-scope vs deferred, ops). For FHIR façades, keep capability-statement.json aligned with runtime.
2. Ensure V2 routes exist (route.v2.xml) so the brief can summarize each route in plain language.
3. From the interface root run:
  - `python3 tools/export_stakeholder_brief.py`
  - (equivalent) `python3 ../../../../tools/export_stakeholder_brief.py --root .` from Sandbox tools/
4. Write a **single** PDF under the interface's **documents/** folder:
  - `documents/<ShortName>_Capability_Brief.pdf`
  - Prefer deriving <ShortName> from the route-diagrams PDF name (`*_V2_Route_Diagrams.pdf` → `*_Capability_Brief.pdf`).
5. Content expectations (generator default): executive summary, who it's for, capabilities, honest boundaries, walkthrough checklist, plain-language how-it-works, security posture, how to see it run, related docs. Prefer **outcome language** over FQCNs.
6. Expose the PDF from the Web UI (generic `/documents/<file>.pdf` is enough; add a stable alias `/documents/capability-brief.pdf` and an Info/Demo link when the UI has chrome).
7. Mention the PDF in README.md **and** announce browser URLs per §6.2 alongside the route diagrams PDF.

Skipping the Capability Brief requires the user to opt out explicitly.

### 6.2 Browser / LAN links for every deliverable PDF (required)

Whenever an agent produces a PDF the user needs to review (route diagrams, research notes, design write-ups, playbooks for an interface, etc.), **do not leave it as a git-only / Finder-only file**. Drive uploads often break when credentials expire — always provide a **clickable browser link** on the LAN.

**Required every time a PDF is created or updated:**

1. Write the PDF under the interface's `documents/` (or `Sandbox docs/` if it is a global playbook).
2. Ensure the Web UI (or a short-lived static server bound to `0.0.0.0`) **serves** that file over HTTP with `Content-Type: application/pdf` and inline disposition (open in browser, not force-download-only).
3. Prefer stable Flask routes under `/documents/...` with `./documents` mounted read-only (pattern: `/documents/route-diagrams.pdf`, `/documents/<name>.pdf`). Add a nav / Routes-tab link when the UI has chrome.
4. With the stack up, **paste both URLs in the agent summary** (and README when durable):
  - Local: `http://localhost:<webui-port>/documents/<file>.pdf`
  - LAN: `http://<192.x.x.x>:<webui-port>/documents/<file>.pdf`
5. Verify with `curl` that the LAN URL returns HTTP 200 and `application/pdf` before calling the work done.
6. Google Drive upload remains optional/extra when the Google Drive secret works — **never** the only way to view the file. If Drive fails, the LAN browser link is still mandatory.

**Anti-pattern:** “PDF is at `documents/Foo.pdf` in the repo” with no running HTTP URL the user can open from a phone or tablet.

### 6.3 SNIP HTML validation report in the Web UI (required when demos write SNIP XML)

When a demo writes SNIP results (`output/snip/*_snip.xml` or similar), the Web UI **must** show a rendered **HTML** report on the “HTML report” tab — not raw / escaped SNIP XML.

**Reference implementation:** `Clients/Demos/edi-837-snip-sqlserver/webui/` (`snip_report.py`, `xslt/*.xslt`, `/api/snip-report`, `iframe #snip-html`).

| Piece                                                                                | Role                                                                                                |
|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>webui/xslt/{transform,normalize,sort,merge,html}.xslt</code>                   | File Fish 14a SNIP Validations Report pipeline Saxon transforms; <b>both args are strings</b>       |
| <code>webui/snip_report.py</code> → <code>build_snip_html(snip_xml, edi_text)</code> | Resolves matching <code>.edi</code> , reads file <code>text</code> , returns <code>text/html</code> |
| GET <code>/api/snip-report?name=...</code>                                           | File list + tabs: <code>iframe</code> for HTML, <code>&lt;pre&gt;</code> for Raw XML only           |
| UI                                                                                   |                                                                                                     |

#### Hard rules:

1. Pass **EDI wire text** (`edi_path.read_text(...)`) into `build_snip_html` — **never** a `pathlib.Path` (that fails the transform and used to dump escaped XML into the `iframe`).
2. On failure, `fallback_html("Report failed: ...")` with a short error — **never** dump the SNIP XML body into the fallback page.
3. Dockerfile must `COPY xslt/ xslt/` (and Saxon via `requirements.txt`).
4. Smoke: `curl /api/snip-report?name=<file>` and confirm the body starts with HTML (`<!DOCTYPE / <html`) and is **not** escaped `&lt;EdiValidationResults`. Add a `tests/plan.json` assertion when the demo ships SNIP.

Copy the SNIP report stack from `edi-837-snip-sqlserver` (or another demo that already passes this check) rather than reimplementing.

### 6.4 XML / XSLT syntax highlighting in the Web UI (required when showing source)

When the Web UI shows XML or XSLT source (XSLT tab, kickout/decision viewers, SNIP raw XML, etc.), use syntax highlighting.



**Shared assets** (copy into each demo's `webui/static/`):

- `Clients/Demos/_shared/webui/code-highlight.js`
- `Clients/Demos/_shared/webui/code-highlight.css`

Load `Highlight.js` (XML) + those assets in `templates/index.html` **before** `app.js`. The helper auto-highlights common viewers (`#xslt-code`, `#xslt-view`, `pre.viewer`, `#snip-view`, ...) when content changes — no per-view rewrite required.

## 6.5 Info tab — review PDFs, ports, and story (required)

Every Sandbox demo Web UI **must** expose an **Info** tab (or equivalent top-level view) with the same layout family used by `edi-837-snip-sqlserver`:

1. Title + short blurb (what the demo does)
2. **Info & review PDFs** list: demo-only note, EIP URL, LAN UI URL, and links to `/documents/capability-brief.pdf`, `/documents/route-diagrams.pdf`, `/documents/test-plan.pdf`, `/documents/test-results.pdf` (plus optional demo-specific PDF links)
3. **Ports** table/list (host ports from compose)
4. Optional extra sections (e.g. SNIP levels) when `DESIGN.md` warrants them

**Shared assets / wiring:**

| Piece            | Path                                                                                                                                                                                                |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Jinja partial    | <code>Clients/Demos/_shared/webui/templates/partials/info_ta</code><br>→ copy to each demo's<br><code>webui/templates/partials/</code>                                                              |
| Document aliases | <code>Clients/Demos/_shared/webui/document_routes.py</code><br>→ copy to each demo's <code>webui/</code> and call<br><code>ensure_document_routes(...)</code> (idempotent;<br>skips existing rules) |
| Apply / refresh  | <code>python3 tools/apply_info_tab_standard.py</code><br>(optional <code>--demo &lt;name&gt;</code> ) — updates<br>metadata-driven blurbs, ports, includes, and<br>bootstrap                        |

In `index.html`:

```
<!-- INFO_TAB_STANDARD:START -->
{% include 'partials/info_tab.html' %}
<!-- INFO_TAB_STANDARD:END -->
```

Flask must provide context (`info_title`, `info_blurb`, `info_note`, `eip_url`, `lan_hint`, `info_ports`, `test_results_pdf`, optional `info_extra_links` / `info_extra_sections`). Prefer a `@app.context_processor` so every page gets them.

**Hard rules:**

1. Do not ship an Info tab that is only LAN/EIP URLs.
2. PDF aliases must return `application/pdf` when the file exists under `documents/` (§6.2).

3. New demos: copy the partial + `document_routes.py` (or re-run the apply script) before claiming Web UI done.
4. After changing shared partial / `document_routes.py`, re-run `tools/apply_info_tab_standard.py` (or sync copies) so demos stay aligned.

**Reference:** `edi-837-snip-sqlserver` (Info chrome + SNIP levels section) and `edi-837-claim-scrub` (review PDF list).

---

## 7. Smoke test (required before “done”)

Minimum bar:

- ☐ `docker compose up -d --build` comes healthy (or known wait documented)
- ☐ One happy-path sample produces **every** expected side effect (SQL row and/or files)
- ☐ Kickout / fail path exercised once **or** explicitly marked “not implemented” in `DESIGN.md`
- ☐ Routes tab renders `route.v2.xml`
- ☐ **Processor Groups** authored (`diagram-groups.json`) for every long chain (§6.1b), or N/A documented when all chains are short
- ☐ **Route design PDF** written under `documents/` with **overview (collapsed groups) + detail pages** (`--config compact` preferred)
- ☐ **Stakeholder Capability Brief PDF** written under `documents/` (`tools/export_stakeholder_brief.py`)
- ☐ **Test Plan PDF** written under `documents/` (`tools/export_test_plan_pdf.py`)
- ☐ **Automated tests run** (`tools/run_interface_tests.py --wait`) with results in `documents/test-results.json / .html / .pdf`
- ☐ **Info tab** shows blurb, EIP/LAN, capability/route/test-plan/test-results PDF links, and ports (§6.5)
- ☐ **Browser/LAN PDF URLs** work (HTTP 200, `application/pdf`) for every review PDF (§6.2)
- ☐ No silent claim of partner-grade validation unless gated

Record: ports, sample used, output paths, LAN PDF links, and any known failures (e.g. SNIP noise).

### 7.1 Living test plan + automated runner (required for every new interface)

Every interface keeps a machine-readable plan and generates a Test Plan PDF next to the other `documents/` deliverables. The same plan is executed on the host so stakeholders and agents see an easy pass/fail list.

**Source of truth:** `tests/plan.json` (JSON). Update this file **as features are built** — new capability new suite/test entries in the same PR/change set.

**Generate the PDF (required when creating/changing an interface):**

```
python3 tools/export_test_plan_pdf.py
# → documents/<ShortName>_Test_Plan.pdf
```

**Run the tests (required before done; also after meaningful route/DESIGN/sample changes):**

```
docker compose up -d --build
./tools/post_up_tests.sh
# or:
python3 tools/run_interface_tests.py --wait
```

**Watch mode (recommended while actively building):**

```
python3 tools/run_interface_tests.py --watch
```

Re-runs when `tests/plan.json`, `DESIGN.md`, routes, samples, SQL, or Web UI sources change.

**Results (easy list):**

| Artifact                       | Where                                                |
|--------------------------------|------------------------------------------------------|
| JSON                           | documents/test-results.json                          |
| HTML list                      | documents/test-results.html                          |
| PDF (open without browser app) | documents/test-results.pdf                           |
| Web UI                         | Tests tab (when present) · GET /api/v2/tests/results |
| Stable PDF alias               | /documents/test-plan.pdf                             |

Announce browser URLs per §6.2 for the Test Plan PDF and results HTML.

**Compose hook:** after every `docker compose up -d --build` for an interface under construction, run `./tools/post_up_tests.sh` (waits for health URLs from the plan, exports the Test Plan PDF if needed, runs tests). Do not claim “done” with failing tests unless the user explicitly accepts them and DESIGN.md Risks records why.

**Test types supported by the shared runner** (`tools/interface_testlib.py`): `http`, `oauth`, `wait`, `file`, `ui` (HTTP + optional Chrome dump-dom), `shell`.

## 8. DESIGN.md template (copy into each interface)

```
# <Interface name> - Design
```

```
## 1. Purpose
<one paragraph>
```

```
## 2. Context / actors
- Sources:
- Destinations:
- Demo vs production:
```

```
## 3. Inbound contract
- Transport:
- Format / envelope:
- Identity fields:
- Samples path:
```

```
## 4. Outbound contract(s)
| Destination | Format | Success criterion |
|-----|-----|-----|
|           |           |           |
```

```
## 5. Pipeline
| Stage | Module / mechanism | Notes |
|-----|-----|-----|
| Listener | | |
| Processors | | |
| Router | | |
| Transports | | |
```

```
## 6. State & idempotency
- Status model:
- When state advances:
- Dedup keys:
```

- Retry / poison:

## ## 7. Validation

- What is checked:
- What is NOT checked:
- Does failure block outbound? (yes/no):

## ## 8. Dual-write / side effects

- Order of commits:
- Compensation:
- Demo shortcuts (if any):

## ## 9. Risks & bottlenecks

| Severity | Risk | Why it bites here | Mitigation / accepted? |
|----------|------|-------------------|------------------------|
| High     |      |                   |                        |
| Med      |      |                   |                        |
| Low      |      |                   |                        |

## ## 10. Ops

- Ports:
- Volumes:
- Heap / special JVM:
- Dependencies / cold start:

## ## 11. Observability

- Logs:
- Kickout dir:
- Transaction / debug tracing: on/off and retention:

## ## 12. Open questions

-

### Risks checklist (seed §9 from these when applicable)

- Claim-before-complete (status flip before all side effects)
- Dual-write without outbox / compensation
- Validation theater (runs but doesn't gate; wrong artifact validated)
- Fake batch split / unused multi-source folders
- Unbounded poll / no throttling into heavy stages
- Overwrite semantics on audit files
- Missing unique keys → duplicates on resubmit
- Runtime vs `eip-root` config drift
- Secrets / PHI on permissive mounts
- Partner transport missing (e.g. no MLLP/ACK when story implies it)
- Hardcoded EDI/HL7 wire text instead of XML + Transformation processor (§3.5)

---

## 9. README.md minimum

Every interface README must include:

1. What it does (bullet flow)
2. Prerequisites (`pilotfish-eip:23R1`, Docker)

3. `docker compose up` instructions
4. Ports table
5. **Local and LAN Web UI URLs** (`http://localhost:<port>/` and `http://192.x.x.x:<port>/`)
6. **Browser links** for review PDFs under `/documents/...` (local + LAN) — see §6.2
7. Smoke / useful commands
8. Explicit **Demo only** callouts for credentials / PHI / validation limits

## 10. Anti-patterns (do not ship without calling out)

| Anti-pattern                                                                                | Do instead                                                                                                                   |
|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Name a stage “Validate” / “SNIP” / “Split” that doesn’t gate or fork                        | Rename or implement the real behavior                                                                                        |
| UPDATE ... PROCESSED in the same SQL that claims work                                       | Claim → process → terminal status                                                                                            |
| Router fan-out with <code>retries=1</code> and no reconcile                                 | Outbox or document as demo risk                                                                                              |
| Listener <code>FileExtensionRestriction</code> mismatched to story samples                  | Align contract + samples + UI                                                                                                |
| Hand-maintained duplicate route trees that disagree                                         | One generator direction; document sync                                                                                       |
| Claiming “HL7 automation” / “SNIP compliant” from heuristic passes                          | Accurate language in README + DESIGN                                                                                         |
| World-writable PHI without demo labeling                                                    | Label + restrict for client work                                                                                             |
| Skipping DESIGN.md because “it’s just a demo”                                               | Short DESIGN.md still required                                                                                               |
| Announcing only <code>localhost</code> after spin-up                                        | Also set <code>LAN_HINT</code> + list<br><code>http://192.x.x.x:&lt;port&gt;/</code> (§1.1)                                  |
| Binding Web UI / publish to <code>127.0.0.1</code> only                                     | Keep default <code>0.0.0.0</code> host publish so LAN devices can connect                                                    |
| Leaving a review PDF only on disk / in git / Drive                                          | Serve via Web UI <code>/documents/...</code> and paste LAN browser URL (§6.2)                                                |
| SNIP “HTML report” tab showing raw/escaped <code>EdiValidationResults</code> XML            | <code>/api/snip-report</code> must pass <b>EDI text</b> into <code>build_snip_html</code> ; iframe shows real HTML (§6.3)    |
| Relying solely on Google Drive for PDF review                                               | Drive is optional; LAN HTTP link is required even when Drive works                                                           |
| Jumping to a custom Java module for auth, HTTP callouts, mapping, or routing                | Prefer PF Call Route / HTTP / Attribute / XSLT first (§3.4); custom only when PF-only is extremely hard                      |
| Hardcoding X12 ( <code>ISA*...</code> ) or HL7 ( <code>MSH ...</code> ) as text in XSLT     | XSLT → PilotFish EDI/HL7 XML → <code>EDITransformationProcessor</code> / <code>HL7TransformationProcessor</code> (§3.5)      |
| Emitting EDI/HL7 without an intermediate structured XML audit in demos                      | Write EDI/HL7 XML to debug/output next to the wire file                                                                      |
| Leaving X12 demos on expired trial tables / basic Segment XML when Sandbox TableData exists | Mount <code>EDI/TableData/x12 + TransactionDataWithVersion</code> → <code>edi-tabledata/&lt;IG&gt;</code> (§3.6)             |
| Hand-rolling sample EDI / duplicate WPC trees outside <code>EDI/TableData</code>            | Prefer <code>EDI/TableData/x12/&lt;IG&gt;/examples</code> (+ XSDs/PDFs); keep story fixtures only when demo logic needs them |

## 11. Definition of done

An interface construct is done when:

1. DESIGN.md exists and §9 Risks is filled (accepted demo risks allowed if labeled).
2. Compose stack runs and smoke test passes.
3. README is enough for a cold start on this machine.
4. Runtime config and diagrams agree **or** drift is documented.
5. Web/route viewer present when routes are part of the deliverable.
6. **Route design PDF** exists under documents/ (overview + group detail pages when §6.1b applies; prefer `--config compact`).
  - 6a. **diagram-groups.json** present for long processor chains (or N/A if all chains are short).
  - 6b. **Stakeholder Capability Brief PDF** exists under documents/ (generated with `tools/export_stakeholder_brief`).
  - 6c. **Test Plan PDF** exists under documents/ and `tests/plan.json` is current.
  - 6d. **Automated test run** completed (`run_interface_tests.py`) with results published under `documents/test-results.*` (pass/fail list + PDF).
7. **LAN URL** is set (`LAN_HINT`) and both localhost + 192.x URLs are listed when the UI is running (§1.1).
8. **Every review PDF** has a working browser link on localhost **and** 192.x (§6.2), pasted in the agent summary.
9. Agent summary lists known limitations in plain language (no compliance theater).
10. **Outbound X12 / HL7** (when applicable) uses structured EDI/HL7 XML + Transformation processors (§3.5)—not hardcoded wire text.
11. **X12 TableData** (when applicable): compose mounts `EDI/TableData/x12` and EDI transforms use enhanced context + `TransactionDataWithVersion` (§3.6).
12. `documents/build-timing.json` exists with `started_at`, `completed_at`, phase durations, `slowest_phases`, and bottlenecks (§4.1); completion summary includes total minutes + top slow phases.
13. **Docker inventory** snapped via `python3 tools/list_sandbox_demo_docker.py` and recorded (`compose_project` + counts); idle stacks called out if relevant (§5.1).

---

## 12. Quick command pack (adapt ports/paths)

```
cd "Clients/Demos/<slug>"
# detect LAN (example); set LAN_HINT in compose before up
LAN_IP=$(ipconfig getifaddr en0) # macOS; expect 192.*
# ensure docker-compose.yml has: LAN_HINT: "http://192.x.x.x:<webui-port>/" (resolved LAN IP)
docker compose up -d --build
echo "Local: http://localhost:<webui-port>/"
echo "LAN:   http://${LAN_IP}<webui-port>/"
docker compose logs -f pilotfish
# happy path: drop sample or use Web UI
ls -la output/*
python3 tools/export_route_diagrams.py --config compact
# Expect overview pages (collapsed Processor Groups) then detail pages per group (§6.1b)
```

```

python3 tools/export_stakeholder_brief.py
python3 tools/export_test_plan_pdf.py
./tools/post_up_tests.sh
# optional: clear GitHub Vault false positives in PDF binaries
python3 ../../../../tools/scrub_pdf_secret_false_positives.py documents/*.pdf 2>/dev/null || true
# ensure PDFs are under documents/
mkdir -p documents && cp -f output/route-diagrams/*Route_Diagrams*.pdf documents/ 2>/dev/null || true
ls -la documents/
# always give the user a browser link (Drive optional / flaky)
echo "Route PDF local: http://localhost:<webui-port>/documents/route-diagrams.pdf"
echo "Brief PDF local: http://localhost:<webui-port>/documents/capability-brief.pdf"
echo "Test plan local: http://localhost:<webui-port>/documents/test-plan.pdf"
echo "Results PDF:      http://localhost:<webui-port>/documents/test-results.pdf"
echo "Results HTML:     http://localhost:<webui-port>/documents/test-results.html"
echo "Route PDF LAN:    http://${LAN_IP}:<webui-port>/documents/route-diagrams.pdf"
echo "Brief PDF LAN:    http://${LAN_IP}:<webui-port>/documents/capability-brief.pdf"
echo "Test plan LAN:    http://${LAN_IP}:<webui-port>/documents/test-plan.pdf"
echo "Results PDF LAN: http://${LAN_IP}:<webui-port>/documents/test-results.pdf"
echo "Results HTML LAN: http://${LAN_IP}:<webui-port>/documents/test-results.html"
curl -sS -o /dev/null -w "%{http_code} %{content_type}\n" \
    "http://${LAN_IP}:<webui-port>/documents/route-diagrams.pdf"
curl -sS -o /dev/null -w "%{http_code} %{content_type}\n" \
    "http://${LAN_IP}:<webui-port>/documents/capability-brief.pdf"
curl -sS -o /dev/null -w "%{http_code} %{content_type}\n" \
    "http://${LAN_IP}:<webui-port>/documents/test-plan.pdf"
# finalize documents/build-timing.json (§4.1), then:
python3 ../../../../tools/list_sandbox_demo_docker.py
docker compose down -v # destroys DB volume - warn user first

```